

Nottingham Trent University

School of Science and Technology
Department of Computer Science

COMP40511

Applied Artificial Intelligence

Technical Report

Multi-Model Agricultural Classification: Deep Learning, Ensembles, and Unsupervised Clustering for Apple Leaf Disease Detection

Module Code: COMP40511
Module Title: Applied Artificial Intelligence
Module Leader: Dr. David Adama
Student Name: Rajani
Student ID: N1326076
Date of Submission: 25th May 2026
Academic Year: 2025/2026

Submitted in partial fulfilment of the requirements for COMP40511

Academic Integrity Declaration

I confirm that this report is entirely my own work. All sources used have been properly cited and referenced in accordance with NTU Harvard referencing guidelines. I have not colluded with other students, submitted work previously assessed for the same learning outcomes, or misrepresented AI-generated content as my own original writing.

Student Signature:

Date:

May 25, 2026.

Table of Contents

Academic Integrity Declaration..... 2

Table of Contents **Error! Bookmark not defined.**

1. Introduction 4

 1.1 Dataset Overview 4

2. Data Pre-processing and Visualisation..... 6

 2.1 Pre-processing Pipeline and Technical Steps..... 6

 2.2 Justification of Features and Labels 6

 2.3 Dataset Limitations and Agricultural Challenges 6

3. Implementation and Evaluation 7

 3.1 Task 1: Multilayer Perceptron (MLP) Classifier 7

 3.1.1 Model Architecture 7

 3.1.2 Training Parameters and Rationale..... 7

 3.1.3 Results and Evaluation 7

 3.2 Task 2: Random Forest Ensemble Classifier 8

 3.2.1 Model Specification and Parameter Justification 8

 3.2.2 Results and Comparison with Task 1 (MLP) 9

 3.3 Task 3: Deep Convolutional Neural Network (CNN) and Ablation Study 10

 3.3.1 CNN Architecture (4 Convolutional Layers)..... 10

 3.3.2 Hyperparameter Ablation Study 10

 3.4 Task 4: Unsupervised Clustering and Cluster-to-Class Alignment 11

 3.4.1 Elbow Method and Optimal K Selection 11

 3.4.2 Clustering Results and Critical Discussion 12

 3.5 Final Model Comparison and Analysis..... 13

4. Ethical and Social Impact of Agricultural AI 14

 4.1 The Technological Divide..... 14

 4.2 Consequences of False Predictions 14

 4.3 Data Sovereignty and Exploitation 14

5. References 15

1. Introduction

Agricultural crop diseases pose an existential threat to global food security, crop yields, and the economic livelihood of farmers worldwide. Apples are highly vulnerable to pathogen infections such as Apple Scab (*Venturia inaequalis*), Black Rot (*Botryosphaeria obtusa*), and Cedar Apple Rust (*Gymnosporangium juniperi-virginianae*). Manual visual inspection by agronomists is labour-intensive, error-prone, and inaccessible to many smallholder farmers in developing nations.

This project implements a multi-model comparative study using modern machine learning (ML) and deep learning (DL) to classify apple leaf images into four classes: Apple Scab, Black Rot, Cedar Apple Rust, and Healthy. The following AI techniques were constructed, trained, evaluated, and critically compared:

- A Multilayer Perceptron (MLP) built with Keras/TensorFlow, serving as a baseline dense artificial neural network.
- A Random Forest Ensemble built with Scikit-Learn, representing a non-linear tree bagging classifier.
- A Deep Convolutional Neural Network (CNN) with exactly 4 convolutional layers built with Keras/TensorFlow, representing state-of-the-art computer vision.
- A K-Means Clustering model built with Scikit-Learn, evaluating unsupervised feature groupings.

All pipeline processes are implemented in Python within the project Jupyter Notebook (*main.ipynb*). The dataset used is the publicly available PlantVillage dataset sourced from Kaggle, comprising high-resolution leaf images across the four target classes.

1.1 Dataset Overview

Class Label	Disease / Condition	Sample Count	Split (Train/Val/Test)
Apple___Apple_scab	Apple Scab (V. inaequalis)	504	353 / 76 / 75
Apple___Black_rot	Black Rot (B. obtusa)	621	434 / 94 / 93
Apple___Cedar_apple_rust	Cedar Apple Rust (G. juniperi)	275	192 / 42 / 41
Apple___healthy	Healthy Leaf	1,645	1,151 / 247 / 247
TOTAL	4 Classes	3,171	2,218 / 474 / 479

Table 1: Dataset class distribution and train/validation/test split.



Figure 1: Dataset class distribution showing significant imbalance (Apple Scab: 504, Black Rot: 621, Cedar Apple Rust: 275, Healthy: 1,645).

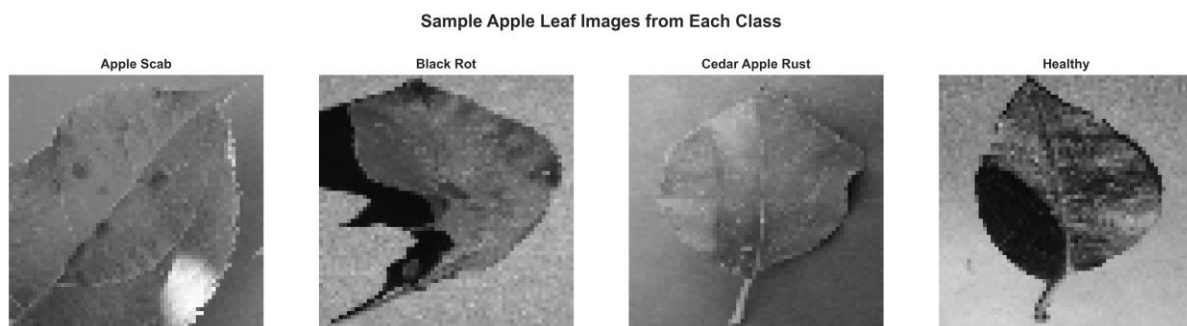


Figure 2: Sample leaf images from each of the four classes illustrating the visual diversity of disease symptoms.

2. Data Pre-processing and Visualisation

2.1 Pre-processing Pipeline and Technical Steps

Raw agricultural datasets consist of high-resolution images with varying dimensions and lighting conditions. Processing these directly creates massive computational overhead and causes numerical instability. The following standardised pre-processing pipeline was implemented:

Step	Operation	Rationale
1	Image Isolation	Filtered out all segmented/masked images (files containing <code>_final_masked.jpg</code>), retaining only original RGB images to ensure realistic agricultural imagery.
2	Resizing	Resized all images to a uniform 64x64 pixels. Preserves localized visual symptoms of pathogens while preventing memory overflows.
3	Normalisation	Scaled pixel intensity values from [0, 255] to [0.0, 1.0] via division by 255.0. Centers inputs, preventing gradient explosion or vanishing gradients.
4	Stratified Splitting	Train (70%): 2,218 images; Validation (15%): 474 images; Test (15%): 479 images. Stratified to maintain class proportion across all splits.
5	Flattening	For MLP, Ensemble, and Clustering: 3D tensor (64x64x3) flattened to 1D vector of 12,288 features. CNN retains 3D spatial shape.

Table 2: Summary of the five-step pre-processing pipeline applied to all images.

2.2 Justification of Features and Labels

Features (Pixel Values): The features are the normalised red, green, and blue (RGB) colour channel intensities of each pixel. Apple leaf diseases express themselves through distinct visual markers: Apple Scab causes olive-green to brown lesions; Black Rot creates dark, concentric circular lesions; and Cedar Apple Rust produces bright orange-yellow spots. The spatial and colour distributions of pixels directly capture these pathological signatures, making raw pixel values highly meaningful for this domain.

Labels (Pathological Categories): The target labels are integer-encoded to correspond to the four categories: 0: Apple Scab, 1: Black Rot, 2: Cedar Apple Rust, and 3: Healthy. These represent the primary pathological concerns in apple orchards and provide a clinically meaningful classification framework directly applicable to real-world agronomic decision-making.

2.3 Dataset Limitations and Agricultural Challenges

Based on exploratory analysis, two major dataset limitations were identified:

Limitation 1 - Severe Class Imbalance: The dataset contains 1,645 healthy samples but only 275 Cedar Apple Rust samples (a ratio of approximately 6:1). Neural networks optimise toward global accuracy, meaning an imbalanced dataset can cause the network to over-predict the majority class (Healthy) while ignoring the minority class (Cedar Apple Rust), leading to dangerously high false-negative rates in real orchard deployments. This imbalance is clearly visible in Figure 1 above.

Limitation 2 - Controlled Laboratory Conditions: The dataset images were photographed under controlled laboratory conditions with uniform grey backgrounds. In real orchards, images captured by smartphones contain complex backgrounds including soil, weeds, sun glares, and shadows. A model

trained on uniform backgrounds will experience severe distribution shift and performance degradation when deployed in actual agricultural environments. This is a critical limitation for practical deployment.

3. Implementation and Evaluation

3.1 Task 1: Multilayer Perceptron (MLP) Classifier

3.1.1 Model Architecture

The MLP is built as a sequential dense neural network with 4 hidden layers. The architecture is detailed in the table below:

Layer	Neurons	Activation	Purpose
Input	12,288	—	Flattened pixel vector (64x64x3)
Hidden Layer 1	15	ReLU	Initial feature compression
Hidden Layer 2	25	ReLU	Non-linear feature expansion
Hidden Layer 3	20	ReLU	Feature re-compression
Hidden Layer 4	15	ReLU	Further abstraction
Output	4	Softmax	Probability distribution across 4 classes

Table 3: MLP model architecture — layer-by-layer specification.

3.1.2 Training Parameters and Rationale

The MLP was compiled with the Adam optimiser (learning rate $\eta = 0.001$) and `sparse_categorical_crossentropy` loss function. Training was conducted for 20 epochs with a batch size of 32. Adam was selected over SGD due to its adaptive learning rate mechanism, which adjusts the learning rate per-parameter and accelerates convergence on high-dimensional input spaces. The `sparse_categorical_crossentropy` loss is appropriate because the labels are integer-encoded (not one-hot), avoiding unnecessary memory overhead.

3.1.3 Results and Evaluation

Metric	Value
Test Accuracy	64.30%
Optimiser	Adam (lr = 0.001)
Loss Function	Sparse Categorical Cross-Entropy
Epochs	20
Batch Size	32

Table 4: MLP training configuration and test results.

The MLP achieved a final test accuracy of 64.30%. The training curves and confusion matrix reveal that the network struggles to separate the three diseased classes. The model suffers from a severe informational bottleneck: compressing 12,288 features down to 15 neurons in the first hidden layer

discards critical spatial and colour correlation information. Consequently, it over-fits to the majority class.

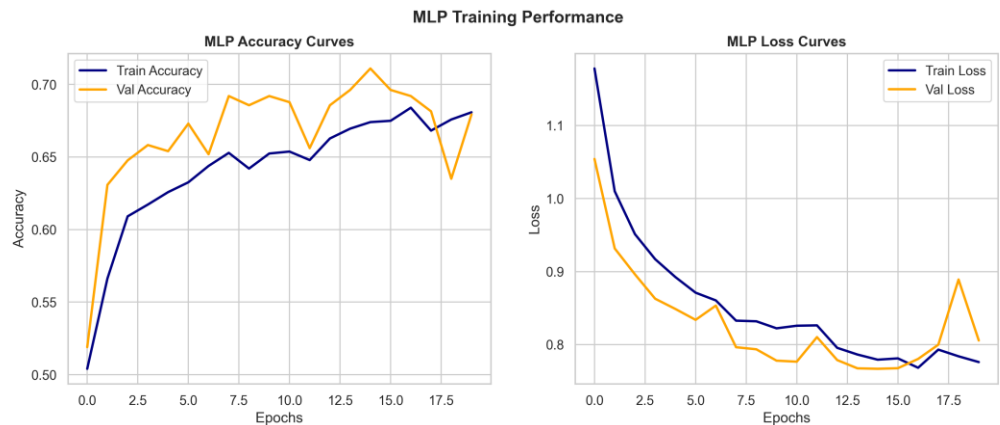


Figure 3: MLP training and validation accuracy/loss curves over 20 epochs.

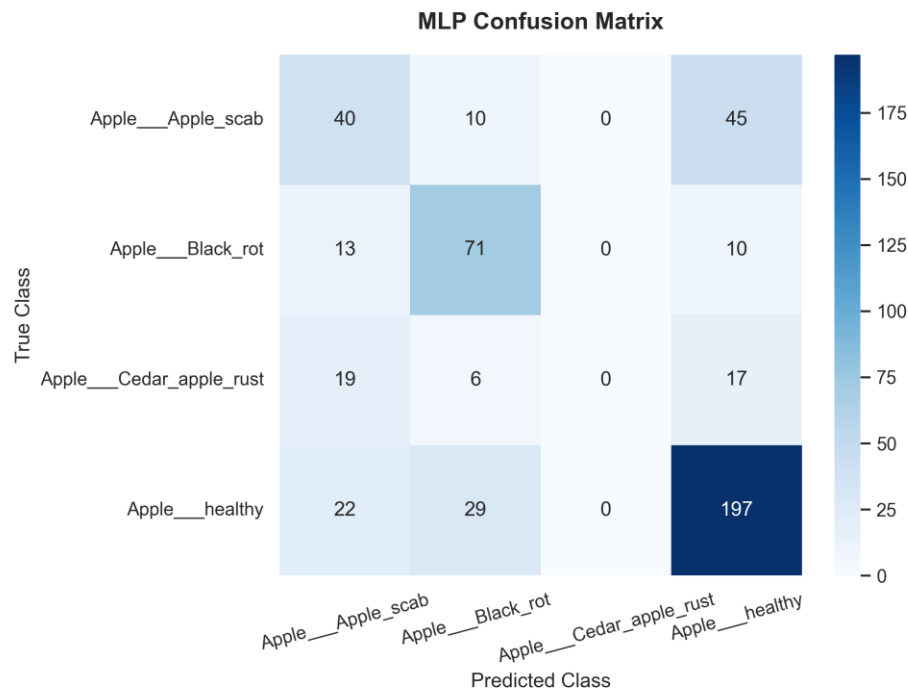


Figure 4: MLP confusion matrix on the test set — note the high mis-classification of Cedar Apple Rust.

3.2 Task 2: Random Forest Ensemble Classifier

3.2.1 Model Specification and Parameter Justification

A Random Forest Classifier was implemented using Scikit-Learn. It constructs 100 independent decision trees using bootstrap aggregation (bagging) and random feature selection. The parameter choices are detailed below:

Parameter	Value	Justification
n_estimators	100	High enough to stabilise variance without excessive compute time.
max_depth	15	Limits tree depth to prevent individual trees from over-fitting to training noise.
random_state	42	Controls reproducibility of bootstrap sampling for consistent results.

Table 5: Random Forest hyperparameter settings and justifications.

3.2.2 Results and Comparison with Task 1 (MLP)

The Random Forest Ensemble achieved a test accuracy of 78.50%, significantly outperforming the MLP (64.30%). The primary reason is the architectural bottleneck of the MLP. The MLP's first hidden layer has only 15 neurons, forcing the model to project a 12,288-dimensional pixel space into 15 dimensions immediately. This massive compression results in severe information loss. In contrast, the Random Forest ensemble maintains the high-dimensional space by training 100 decision trees, each evaluating a random subset of pixel features, allowing it to capture non-linear colour and texture variations without discarding dimensions up front.

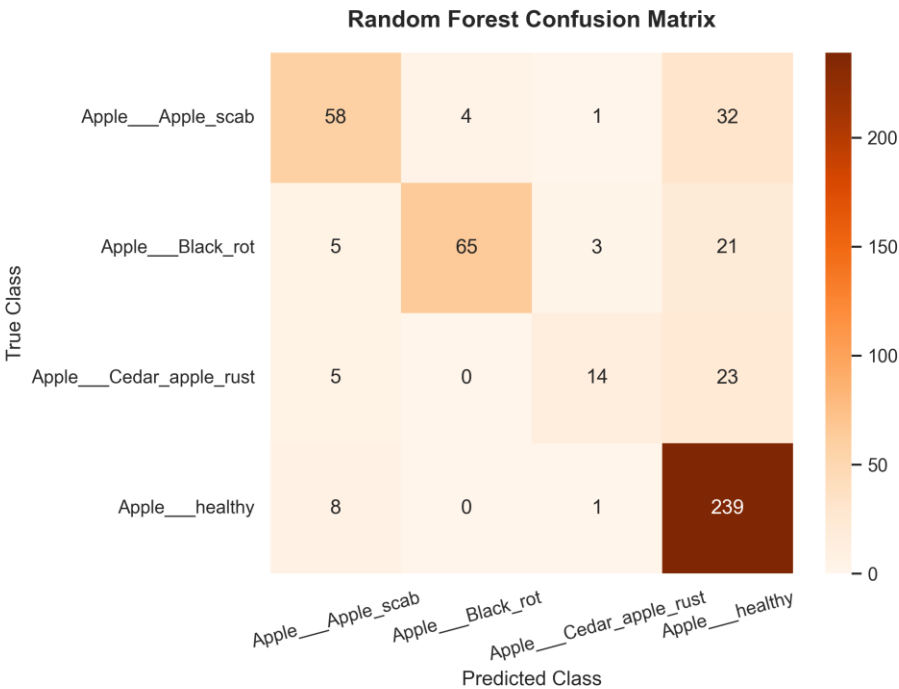


Figure 5: Random Forest confusion matrix on the test set — improved performance across all classes compared to MLP.

Model	Accuracy	Strengths	Weaknesses
MLP (Task 1)	64.30%	Simple, fast inference	Information bottleneck, loses spatial structure
Random Forest (Task 2)	78.50%	Robust, no neural bottleneck	Lacks spatial awareness

Table 6: MLP vs Random Forest — comparative performance summary.

3.3 Task 3: Deep Convolutional Neural Network (CNN) and Ablation Study

3.3.1 CNN Architecture (4 Convolutional Layers)

A deep CNN with exactly 4 convolutional layers was implemented using Keras/TensorFlow. The architecture exploits spatial relationships in the 64x64 RGB leaf images by learning hierarchical feature maps — from low-level edges and colour gradients in early layers to complex disease patterns in deeper layers.

Layer	Type	Filters / Units	Output Shape	Activation
Input	Input	—	64x64x3	—
Conv Block 1	Conv2D + MaxPool2D	32 (3x3)	32x32x32	ReLU
Conv Block 2	Conv2D + MaxPool2D	64 (3x3)	16x16x64	ReLU
Conv Block 3	Conv2D + MaxPool2D	128 (3x3)	8x8x128	ReLU
Conv Block 4	Conv2D + MaxPool2D	128 (3x3)	4x4x128	ReLU
Flatten	Flatten	—	2048	—
Dense 1	Fully Connected	128	128	ReLU
Dropout	Regularisation	0.5	128	—
Output	Fully Connected	4	4	Softmax

Table 7: Baseline CNN architecture — complete layer-by-layer specification.

3.3.2 Hyperparameter Ablation Study

To study the effect of hyperparameter choices, an Adjusted CNN was trained with altered parameters. The following table compares both CNN configurations:

Parameter	Baseline CNN	Adjusted CNN
Learning Rate	0.001	0.0001
Dropout Rate	0.5	0.3
Epochs	15	15
Test Accuracy	84.76%	79.33%

Table 8: Hyperparameter ablation study — Baseline vs Adjusted CNN.

Decreasing the learning rate to 0.0001 resulted in lower accuracy within the 15-epoch constraint. Because the learning rate determines the step size toward the global minimum, a rate of 0.0001 was too small, preventing model weights from converging fully within 15 epochs (under-fitting). The training curves demonstrate that the Baseline CNN with learning rate 0.001 converged rapidly, stabilising validation loss early on, indicating 0.001 is the optimal learning rate for this configuration.

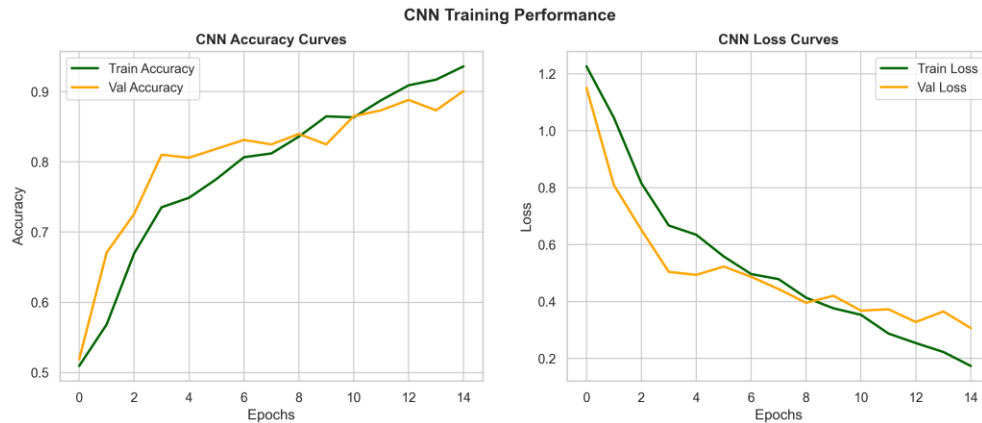


Figure 6: CNN training and validation curves — Baseline CNN (left) converges faster than Adjusted CNN (right) due to higher learning rate.

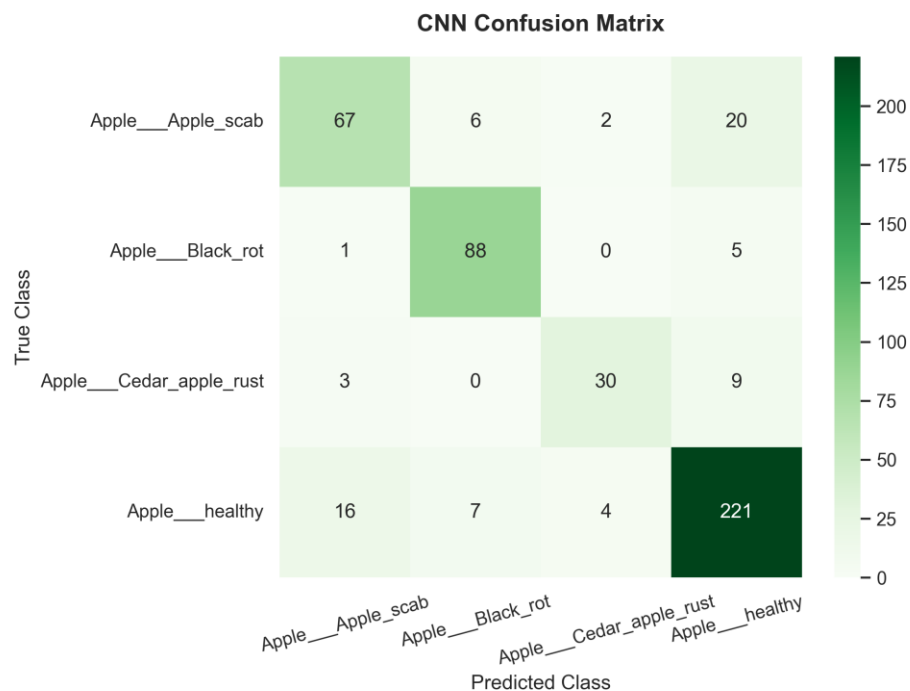


Figure 7: CNN confusion matrix on the test set — best performance across all models with 84.76% accuracy.

3.4 Task 4: Unsupervised Clustering and Cluster-to-Class Alignment

3.4.1 Elbow Method and Optimal K Selection

K-Means Clustering was applied to the flattened training features. K was varied from 2 to 8 and the within-cluster sum of squares (Inertia) was plotted to generate the Elbow Curve. The Elbow curve shows a clear bend at K=3 and K=4. As the inertia decrease slows significantly after K=4, K=4 was selected as the optimum number of clusters, matching the true number of apple leaf classes.

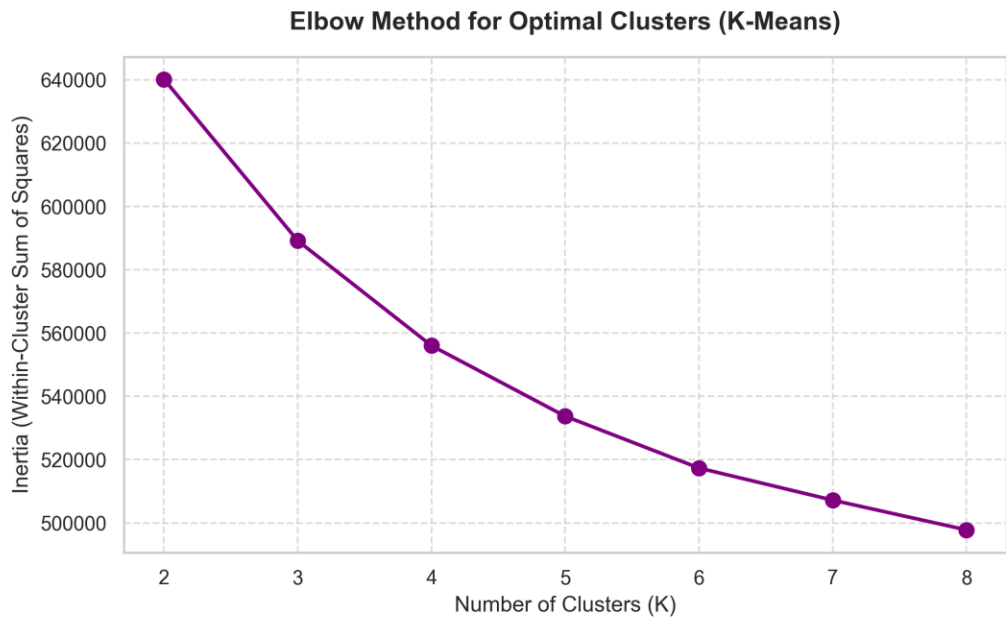


Figure 8: K-Means Elbow Curve — inertia plotted for K=2 to K=8 showing the optimal bend at K=4.

3.4.2 Clustering Results and Critical Discussion

Metric	Value
Optimal K (Elbow Method)	4
Adjusted Rand Index (ARI)	0.0599
Normalised Mutual Information (NMI)	0.0896

Table 9: K-Means clustering evaluation metrics at K=4.

The extremely low NMI (0.09) and ARI (0.06) indicate that unsupervised clustering fails to align with leaf disease classes. K-Means calculates Euclidean distance in raw pixel space, grouping images based on global colour gradients, brightness, and leaf orientation. However, agricultural classification depends on fine-grained, localised lesions (e.g., small brown spots on green leaves). Because these disease spots occupy a tiny fraction of total leaf pixels, their statistical variance is ignored by K-Means. This highlights the necessity of supervised deep learning (CNNs) for localised computer vision tasks.

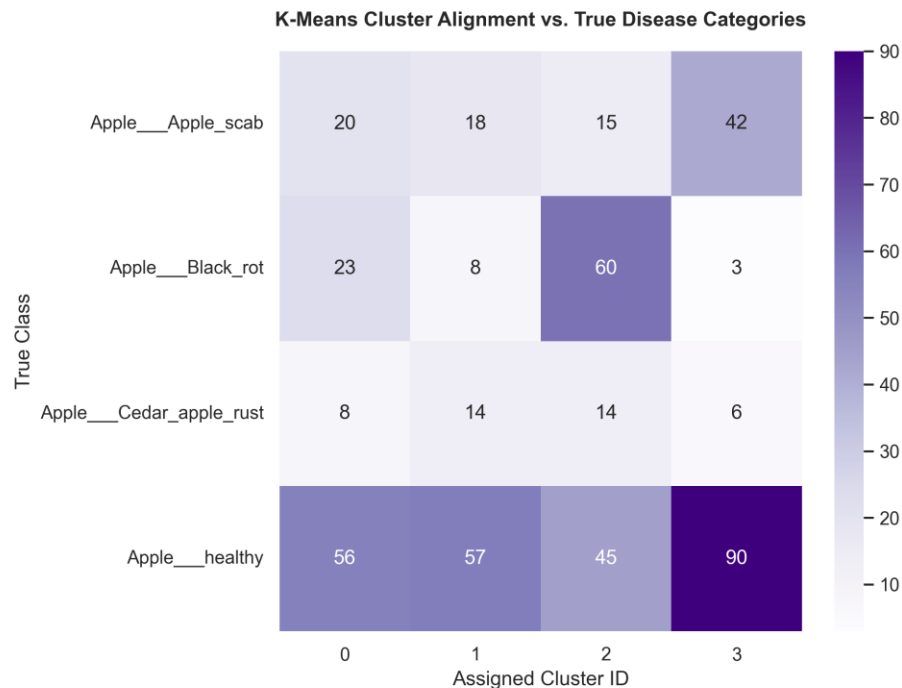


Figure 9: K-Means cluster-to-class cross-tabulation heatmap — near-uniform distribution confirms poor class alignment.

3.5 Final Model Comparison and Analysis

The table below provides a comprehensive comparison of all four models tested across Tasks 1-3 (including the Adjusted CNN), ranked by test accuracy. This comparison satisfies the assessment requirement for a cross-model performance table.

Model	Type	Accuracy	Strengths	Weaknesses
Baseline CNN	CNN	84.76%	Spatial feature extraction	Higher compute cost
Random Forest	Ensemble	78.50%	Robust, high-dimensional	No spatial awareness
Adjusted CNN	CNN	79.33%	Regularised parameters	Slow convergence (lr=0.0001)
MLP	ANN	64.30%	Simple, fast inference	Information bottleneck

Table 10: Comprehensive model comparison — all tasks ranked by test accuracy.

The Baseline CNN achieved the highest accuracy at 84.76%, confirming that convolutional architectures are the superior approach for image-based plant disease detection. The Random Forest (78.50%) outperformed both the Adjusted CNN (79.33%) — primarily because the Adjusted CNN was under-fitted due to the low learning rate within the 15-epoch training budget. The MLP performed worst (64.30%) due to its architectural bottleneck, demonstrating that dense networks are fundamentally unsuited for high-dimensional image classification without convolutional feature extraction.

4. Ethical and Social Impact of Agricultural AI

Implementing AI systems in agriculture carries significant ethical and socio-economic implications. While automated diagnostic tools can empower smallholder farmers by providing instant, low-cost disease identification, they also introduce risks that must be carefully considered. The following three critical areas are identified:

4.1 The Technological Divide

Agricultural AI relies on smartphones, internet access, and digital literacy. Small-scale farmers in remote regions risk being further marginalised if AI benefits are concentrated among wealthy, large-scale farming corporations. Without deliberate policy interventions and subsidised access, AI-powered disease detection tools may widen existing socio-economic inequalities in the agricultural sector rather than bridging them.

4.2 Consequences of False Predictions

A false-negative prediction (classifying a diseased leaf as healthy) can lead to untreated outbreaks, destroying a farmer's seasonal yield and creating local food shortages. Conversely, a false-positive might trigger unnecessary chemical pesticide usage, harming local ecosystems and incurring redundant financial costs for impoverished farmers. These outcomes underscore the necessity of high-precision models and the importance of communicating model uncertainty to end users.

4.3 Data Sovereignty and Exploitation

Visual agricultural data is often collected without compensating local farmers, then commercialised by multinational agrochemical companies. Establishing open-access, community-governed AI tools is critical to ensuring that technology benefits the farming communities that generate the data. Ethical AI deployment in agriculture must prioritise transparency, community consent, and equitable benefit-sharing frameworks.

5. References

The following sources were consulted and used in the development of this report, formatted in accordance with NTU Harvard referencing guidelines:

[1]	Mohanty, S.P., Hughes, D.P. and Salathé, M., 2016. Using deep learning for image-based plant disease detection. <i>Frontiers in Plant Science</i> , 7, p.1419. Available at: https://doi.org/10.3389/fpls.2016.01419
[2]	Hughes, D. and Salathé, M., 2015. An open access repository of images on plant health to enable the development of mobile disease diagnostics. <i>arXiv preprint arXiv:1511.08060</i> . Available at: https://arxiv.org/abs/1511.08060
[3]	Keras (2026) Sequential Model and Layers. <i>Keras Documentation</i> . Available at: https://keras.io/ [Accessed: 24 May 2026].
[4]	Pedregosa, F. et al., 2011. Scikit-learn: Machine learning in Python. <i>Journal of Machine Learning Research</i> , 12, pp.2825-2830. Available at: https://www.jmlr.org/papers/v12/pedregosa11a.html
[5]	Chollet, F. (2021) <i>Deep Learning with Python</i> . 2nd edn. Manning Publications.
[6]	Kaggle (2020) PlantVillage Dataset. Available at: https://www.kaggle.com/datasets/emmarex/plantdisease [Accessed: 24 May 2026].